DSA1604-KNOWLEDGE ENGINEERING

S.CHANDRA SHEKHAR REDDY

192372164

LAB EXPERIMENTS

EXP-1:Knowledge Representation Using Python

AIM: To represent facts about a student using a Python dictionary and display the stored knowledge.

Software Required: Python 3.x

Procedure / Algorithm:

1. Create a dictionary containing student attributes.
2. Traverse the dictionary using items().
3. Display each attribute and its value.

Program

```
Student = {
&amp;quot;name&amp;quot;: &amp;quot;Arun&amp;quot;,
&amp;quot;department&amp;quot;: &amp;quot;CSE&amp;quot;,
&amp;quot;course&amp;quot;: &amp;quot;Python Programming&amp;quot;,
&amp;quot;year&amp;quot;: 2
}
print(&amp;quot;Student Knowledge Representation&amp;quot;)
for key, value in student.items():
print(key, &amp;quot;:&amp;quot;, value)
```

OUTPUT:

main.py

```
1
2  student = {
3      "name": "Arun",
4      "department": "CSE",
5      "course": "Python Programming",
6      "year": 2
7  }
8
9  print("Student Knowledge Representation")
10
11 for key, value in student.items():
12     print(key, ":", value)
13
```

```
Student Knowledge Representation
name : Arun
department : CSE
course : Python Programming
year : 2
```

RESULT: Knowledge was represented as structured facts and displayed successfully.

EXP-2 Knowledge Acquisition

Aim: To acquire knowledge from a user and store it as structured facts.

Software Required: Python 3.x

Procedure / Algorithm

4. Read the expert's name, domain, and years of experience.

5. Store the values in a dictionary.

6. Display the acquired knowledge.

Program

```
name = input(&quot;Enter expert name: &quot;)
domain = input(&quot;Enter area of expertise: &quot;)
experience = int(input(&quot;Enter years of experience: &quot;))
knowledge = {
&quot;expert&quot;: name,
&quot;domain&quot;: domain,
&quot;experience&quot;: experience
}
print(&quot;\nAcquired Knowledge&quot;)
print(&quot;Expert:&quot;, knowledge[&quot;expert&quot;])
print(&quot;Domain:&quot;, knowledge[&quot;domain&quot;])
print(&quot;Experience:&quot;, knowledge[&quot;experience&quot;], &quot;years&quot;)
```

OUTPUT:

main.py

```python

name = input("Enter expert name: ")
domain = input("Enter area of expertise: ")
experience = int(input("Enter years of experience: "))

knowledge = {
    "expert": name,
    "domain": domain,
    "experience": experience
}

print("\nAcquired Knowledge")
print("Expert:", knowledge["expert"])
print("Domain:", knowledge["domain"])
print("Experience:", knowledge["experience"], "years")

```

```
Enter expert name: knowledge
Enter area of expertise: domain
Enter years of experience: 2

Acquired Knowledge
Expert: knowledge
Domain: domain
Experience: 2 years
```

RESULT: Knowledge was collected from the user and stored in a structured form.

EXP-3 Knowledge-Based Course Recommendation System

Aim: To recommend a course using a small knowledge base and decision rules.

Software Required: Python 3.x

Procedure / Algorithm

7. Read the student's area of interest.

8. Compare the interest with predefined conditions.

9. Display the recommended course or a fallback message.

Program

interest = input("Enter your interest (AI, Data, Web): ").strip().lower()

if interest == "ai":

course = "Artificial Intelligence"

elif interest == "data":

course = "Data Science"

elif interest == "web":

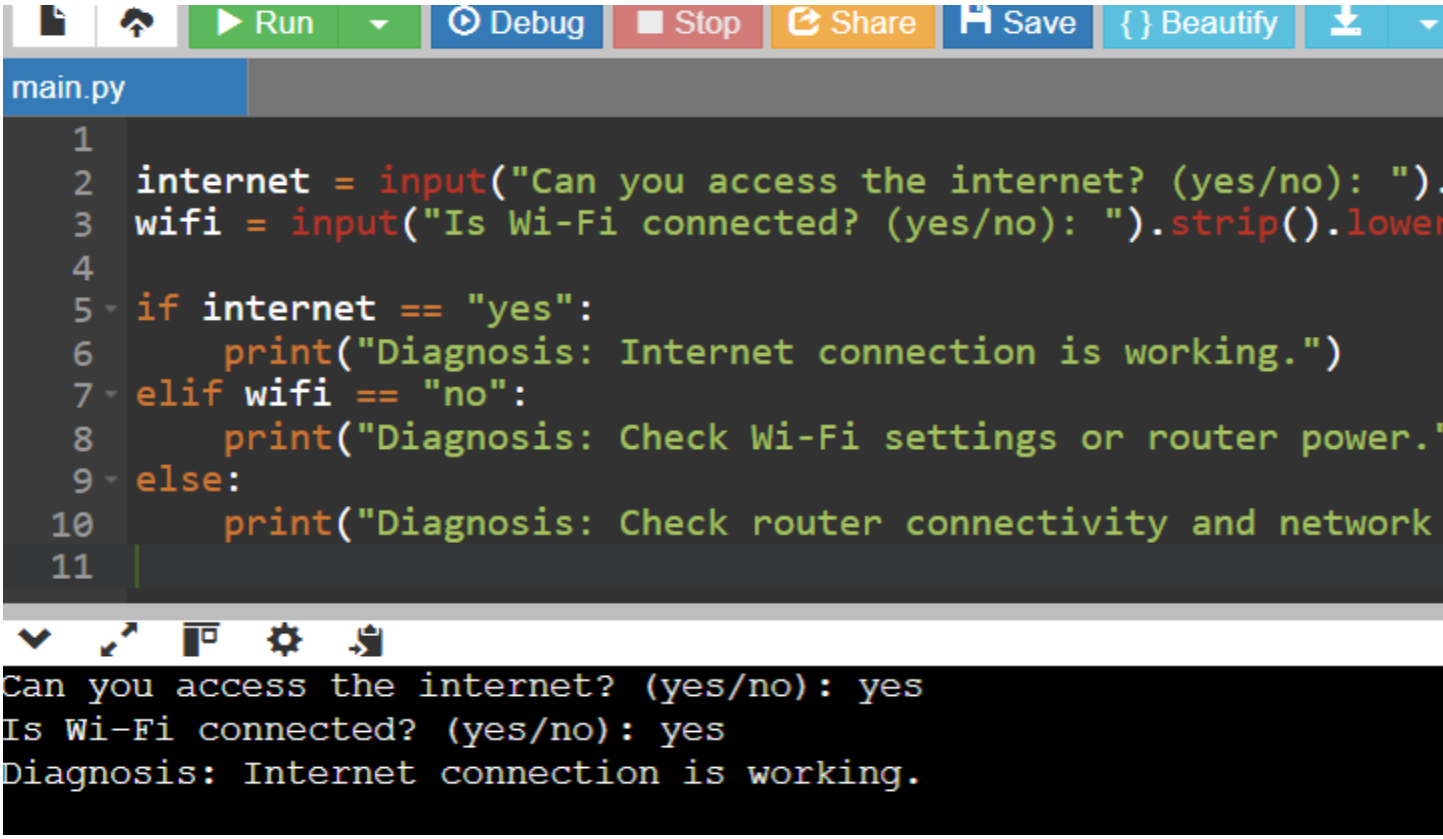
course = "Web Development"

else:

course = "No matching course found"

print("Recommended Course:", course)

OUTPUT:



RESULT: The system used stored knowledge and rules to produce a recommendation.

Experiment 4: Rule-Based Expert System for Network Troubleshooting

Aim: To infer a possible network troubleshooting action from user-provided symptoms.

Software Required: Python 3.x

Procedure / Algorithm

22. Ask whether internet access is available.

23. Ask whether Wi-Fi is connected.

24. Apply IF-THEN rules to display a possible troubleshooting step.

Program

internet = input("Can you access the internet? (yes/no): ").strip().lower()

wifi = input("Is Wi-Fi connected? (yes/no): ").strip().lower()

if internet == "yes":

print("Diagnosis: Internet connection is working.")

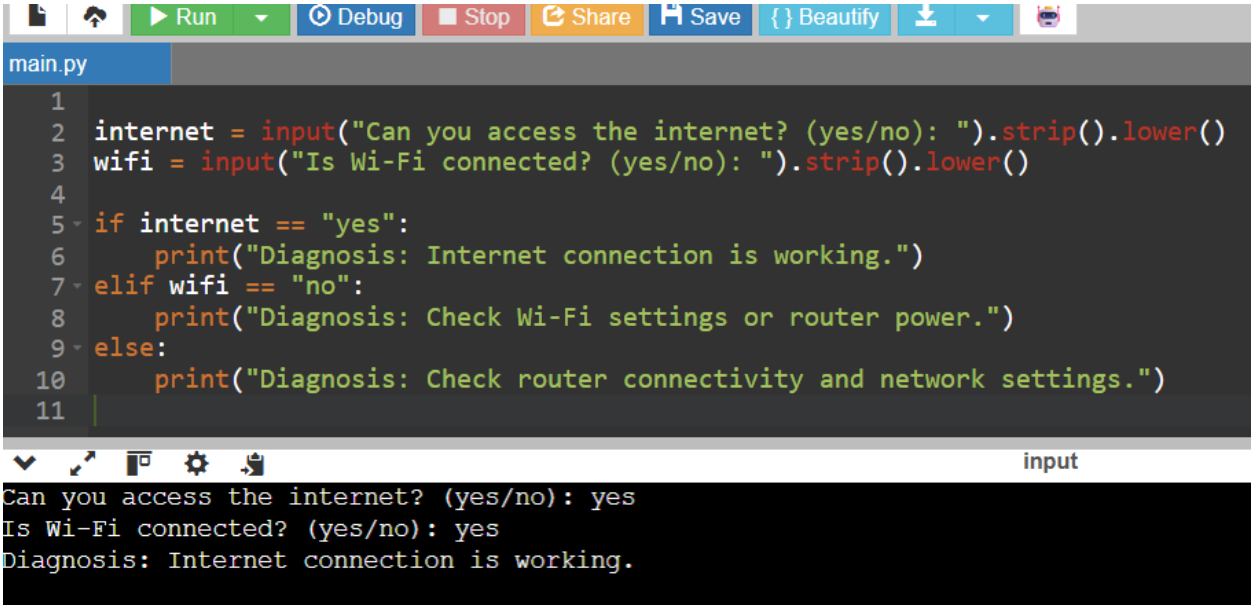
elif wifi == "no":

print("Diagnosis: Check Wi-Fi settings or router power.")

else:

print("Diagnosis: Check router connectivity and network settings.")

Output:



Result: The rule-based system selected a troubleshooting suggestion. This is a teaching demonstration, not a professional diagnostic tool.

Experiment 5: Semantic Network Using NetworkX

Aim: To represent concepts and labeled relationships as a directed semantic network.

Software Required: Python 3.x and NetworkX (install with: pip install networkx)

Procedure / Algorithm

25. Create a directed graph.

26. Add concepts as nodes implicitly through edges.

27. Add labeled relationships.

28. Display each relationship.

Program

```
import networkx as nx

G = nx.DiGraph()

G.add_edge(&amp;quot;Dog&amp;quot;, &amp;quot;Mammal&amp;quot;, relation=&amp;quot;is-a&amp;quot;)

G.add_edge(&amp;quot;Mammal&amp;quot;, &amp;quot;Animal&amp;quot;, relation=&amp;quot;is-a&amp;quot;)

G.add_edge(&amp;quot;Dog&amp;quot;, &amp;quot;Tommy&amp;quot;, relation=&amp;quot;has-instance&amp;quot;)

print(&amp;quot;Semantic Network Relationships:&amp;quot;)

for source, target, data in G.edges(data=True):

print(source, &amp;quot;--&amp;quot;, data[&amp;quot;relation&amp;quot;], &amp;quot;--&amp;gt;&amp;quot;, target)
```

Output:

```python
import networkx as nx

knowledge_graph = nx.DiGraph()

knowledge_graph.add_edge("Arun", "Python", relation="studies")
knowledge_graph.add_edge("Arun", "Computer Science", relation="belongs_to")
knowledge_graph.add_edge("Python", "Programming Language", relation="is_a")

print("Knowledge Graph Triples:")

for subject, obj, data in knowledge_graph.edges(data=True):
    print(subject, "--", data["relation"], "-->", obj)
```

input

```
Knowledge Graph Triples:
Arun -- studies --> Python
Arun -- belongs_to --> Computer Science
Python -- is_a --> Programming Language
```

Result: The semantic network represented concepts and their relationships.

Experiment 6: Frame-Based Knowledge Representation

Aim: To represent an object as a frame containing slots and values.

Software Required: Python 3.x

Procedure / Algorithm

17. Create a dictionary to represent a frame.

18. Use keys as slots and values as slot values.

19. Display the frame.

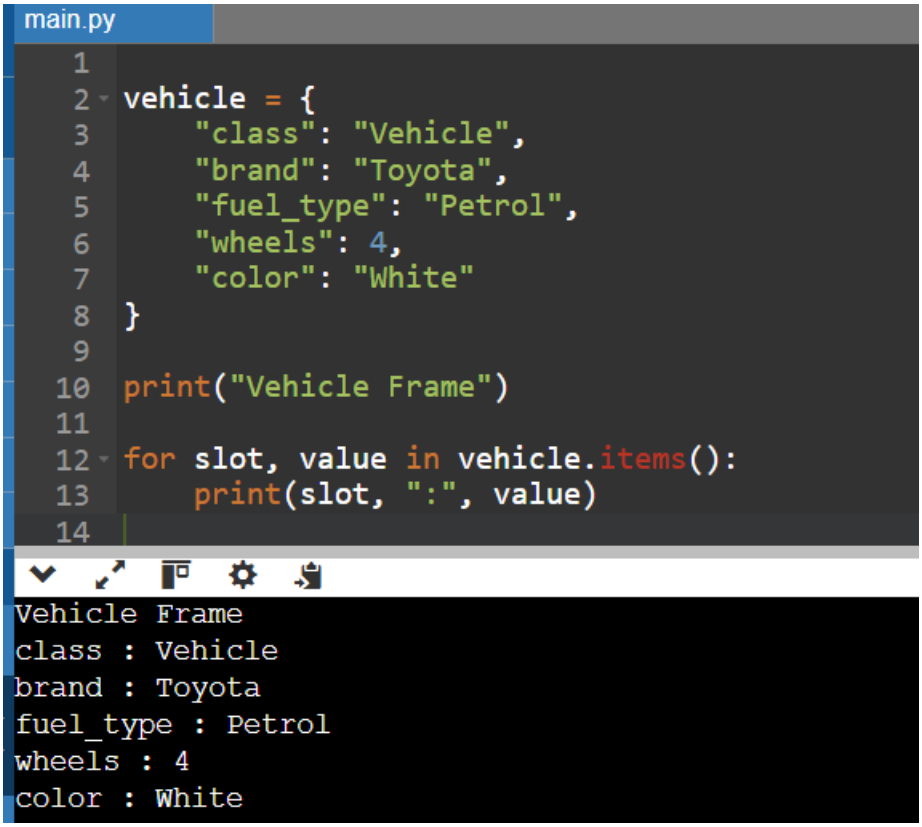
Program

```
vehicle = {
    "class": "Vehicle",
    "brand": "Toyota",
```

```
    "fuel_type": "Petrol",
    "wheels": 4,
    "color": "White"
}

print("Vehicle Frame")
for slot, value in vehicle.items():
    print(slot, ":", value)
```

Output:



Result: The vehicle was represented using slots and their corresponding values.

Experiment 8: Ontology Development Using Protégé

Aim: To create a simple university ontology with classes, properties, and individuals.

Software Required: Protégé Desktop and a compatible OWL reasoner such as HermiT

Procedure / Algorithm

24. Create a new ontology named UniversityOntology.

25. Create classes Person, Student, Faculty, and Course.

26. Make Student and Faculty subclasses of Person.

27. Create the object property enrolledIn with domain Student and range Course.

28. Create individuals Arun (Student) and PythonProgramming (Course).

29. Assert that Arun enrolledIn PythonProgramming.

30. Run the reasoner and inspect inferred class membership.

Implementation Steps

- Create a new ontology named UniversityOntology.
- Create classes Person, Student, Faculty, and Course.
- Make Student and Faculty subclasses of Person.
- Create the object property enrolledIn with domain Student and range Course.
- Create individuals Arun (Student) and PythonProgramming (Course).
- Assert that Arun enrolledIn PythonProgramming.
- Run the reasoner and inspect inferred class membership.

Sample Input

Classes:

Person

Student

Faculty

Course

Individuals:

Arun : Student

PythonProgramming : Course

Object property:

Arun enrolledIn PythonProgramming

Output;

```python
# Define classes
classes = ["Person", "Student", "Faculty", "Course"]

# Define individuals and their classes
individuals = {
    "Arun": "Student",
    "PythonProgramming": "Course"
}

# Define object property
object_properties = [
    ("Arun", "enrolledIn", "PythonProgramming")
]

print("Classes:")
for c in classes:
    print(c)

print("\nIndividuals:")
for name, cls in individuals.items():
    print(f"{name} : {cls}")

print("\nObject Properties:")
for subject, relation, obj in object_properties:
    print(f"{subject} {relation} {obj}")
```

```
Classes:
Person
Student
Faculty
Course

Individuals:
Arun : Student
PythonProgramming : Course

Object Properties:
Arun enrolledIn PythonProgramming
```

Results: Created a university ontology and used subclass reasoning to infer that Arun is a Person.

Experiment 7: Production Rules and Forward Chaining

Aim: To implement IF-THEN production rules and infer new facts through forward chaining.

Software Required: Python 3.x

Procedure / Algorithm

20. Initialize a set of facts.

21. Define rules as condition sets and conclusions.

22. Repeatedly apply rules whose conditions are satisfied.

23. Stop when no new facts are inferred.

Program

```
facts = {"has_fever", "has_cough"}


rules = [
    ({"has_fever", "has_cough"}, "possible_flu"),
    ({"possible_flu"}, "recommend_medical_review")
]


changed = True


while changed:
    changed = False


    for conditions, conclusion in rules:
        if conditions.issubset(facts) and conclusion not in facts:
            facts.add(conclusion)
            print("Inferred:", conclusion)
            changed = True


print("\nFinal Facts:")
for fact in sorted(facts):
print(fact)
```

Output:

main.py

```python

facts = {"has_fever", "has_cough"}

rules = [
    ({"has_fever", "has_cough"}, "possible_flu"),
    ({"possible_flu"}, "recommend_medical_review")
]

changed = True

while changed:
    changed = False

    for conditions, conclusion in rules:
        if conditions.issubset(facts) and conclusion not in facts:
            facts.add(conclusion)
            print("Inferred:", conclusion)
            changed = True

print("\nFinal Facts:")

for fact in sorted(facts):
    print(fact)

```

input

```
Inferred: recommend_medical_review

Final Facts:
has_cough
has_fever
possible_flu
recommend_medical_review
```

Results: Forward chaining derived additional facts by applying production rules. The medical-themed rules are illustrative only.

Experiment 9: Knowledge Graph Construction Using Python

Aim: To construct a knowledge graph of entities and relationships using a directed graph**.**

Software Required: Python 3.x and NetworkX (install with: pip install networkx)

Procedure / Algorithm

31. Create a directed graph.

32. Add entity relationships as labeled edges.

33. Print the subject, relation, and object for each edge.

Program:

```
import networkx as nx

knowledge_graph = nx.DiGraph()

knowledge_graph.add_edge("Arun", "Python", relation="studies")
knowledge_graph.add_edge("Arun", "Computer Science", relation="belongs_to")
knowledge_graph.add_edge("Python", "Programming Language", relation="is_a")

print("Knowledge Graph Triples:")

for subject, obj, data in knowledge_graph.edges(data=True):
    print(subject, "--", data["relation"], "-->", obj)
```

Output:

```python
import networkx as nx

knowledge_graph = nx.DiGraph()

knowledge_graph.add_edge("Arun", "Python", relation="studies")
knowledge_graph.add_edge("Arun", "Computer Science", relation="belongs_to")
knowledge_graph.add_edge("Python", "Programming Language", relation="is_a")

print("Knowledge Graph Triples:")

for subject, obj, data in knowledge_graph.edges(data=True):
    print(subject, "--", data["relation"], "-->", obj)
```

input

```
Knowledge Graph Triples:
Arun -- studies --> Python
Arun -- belongs_to --> Computer Science
Python -- is_a --> Programming Language
```

Results: The graph represented entities as nodes and relationships as labeled edges.

Experiment 10: Knowledge Management System

Aim: To store and retrieve frequently asked questions and answers using a small searchable knowledge base.

Software Required: Python 3.x

Procedure / Algorithm

34. Create a dictionary of topics and answers.

35. Accept a search topic from the user.

36. Display the matching answer or a not-found message.

Program

```
knowledge_base = {
    "python": "Python is a high-level programming language.",
```

```
    "ontology": "An ontology formally defines concepts and relationships.",
    "knowledge graph": "A knowledge graph connects entities through relationships.",
    "expert system": "An expert system uses rules and knowledge to solve problems."
}

print("Knowledge Management System")
print("Available topics:", ", ".join(knowledge_base.keys()))

query = input("\nEnter a topic to search: ").strip().lower()

if query in knowledge_base:
    print("Knowledge Found:")
    print(knowledge_base[query])
else:
    print("No information found for this topic.")
```

Output:

main.py

```python

knowledge_base = {
    "python": "Python is a high-level programming language.",
    "ontology": "An ontology formally defines concepts and relationships.",
    "knowledge graph": "A knowledge graph connects entities through relationships.",
    "expert system": "An expert system uses rules and knowledge to solve problems."
}

print("Knowledge Management System")
print("Available topics:", ", ".join(knowledge_base.keys()))

query = input("\nEnter a topic to search: ").strip().lower()

if query in knowledge_base:
    print("Knowledge Found:")
    print(knowledge_base[query])
else:
    print("No information found for this topic.")

```

input

```
Knowledge Management System
Available topics: python, ontology, knowledge graph, expert system

Enter a topic to search: ontology
Knowledge Found:
An ontology formally defines concepts and relationships.
```

Results: The program stored and retrieved knowledge using a searchable dictionary.

Experiment 11: RDF Knowledge Representation Using RDFLib

Aim: To represent facts as Resource Description Framework triples.

Software Required: Python 3.x and RDFLib (install with: pip install rdflib)

Procedure / Algorithm

37. Create an RDF graph and namespace.

38. Add subject-predicate-object triples.

39. Iterate through the graph and print the triples.

Program

```
from rdflib import Graph, Namespace, Literal


g = Graph()
```

EX = Namespace("http://example.org/")

g.add((EX.Arun, EX.studies, EX.Python))

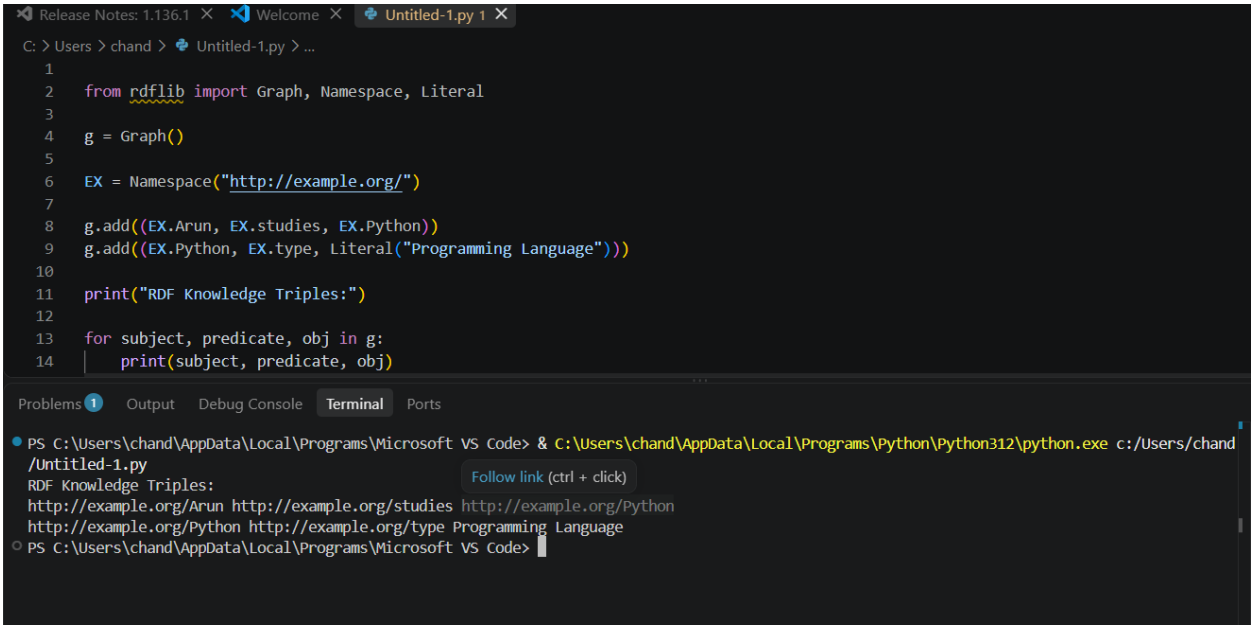
g.add((EX.Python, EX.type, Literal("Programming Language")))

print("RDF Knowledge Triples:")

for subject, predicate, obj in g:

    print(subject, predicate, obj)

Output:



Results: Knowledge was represented as RDF triples. The order of displayed triples may vary.

Experiment 12: Knowledge Graph Traversal Using NetworkX

Aim: To find a path between two entities in a knowledge graph using breadth-first search.

Software Required: Python 3.x and NetworkX (install with: pip install networkx)

Procedure / Algorithm

40. Create a graph of related concepts.

41. Specify the starting and target entities.

42. Use shortest_path to find a connection.

43. Display the path or a message if no path exists.

Program

```
import networkx as nx

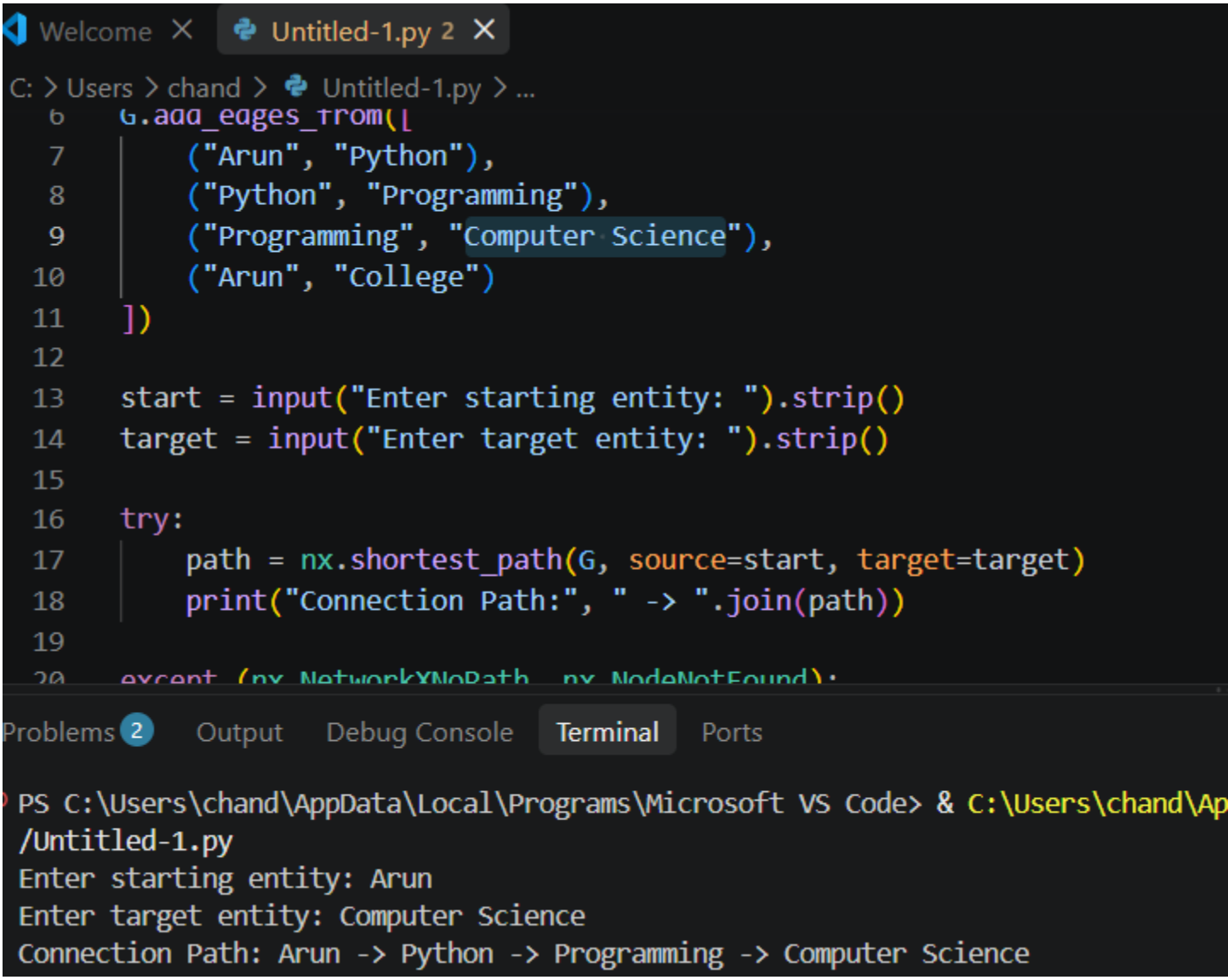
G = nx.Graph()

G.add_edges_from([
    ("Arun", "Python"),
    ("Python", "Programming"),
    ("Programming", "Computer Science"),
    ("Arun", "College")
])

start = input("Enter starting entity: ").strip()
target = input("Enter target entity: ").strip()

try:
    path = nx.shortest_path(G, source=start, target=target)
    print("Connection Path:", " -> ".join(path))
except (nx.NetworkXNoPath, nx.NodeNotFound):
    print("No path found between the given entities.")
```

Output:



Results: The program found a path connecting the two entities in the knowledge graph.